Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль №1

«Связи и запросы вариант 22 А»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Сердюков А.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Реализовать обработку связанных коллекций на Python. Вариант предметной области 22: Библиотека и Язык программирования. Используется набор запросов А. Программа выполняет соединения один ко многим и многие ко многим, сортировку и агрегирование.

Модель данных

Library хранит id, name, module_count и language_id. Количество модулей используется как количественный признак. Language хранит id и name. LibraryLanguage содержит library_id и language_id и реализует отдельную связь M:N. Основной язык в Library задаёт учебное распределение для связи 1:N.

Созданы 4 библиотеки (Polars, NumPy, Boost, serde), 3 языка (Python, Rust, C++) и 5 записей связи. Число модулей 12, 8, 6, 4 задано условно для учебного примера. Polars связан с Python и Rust; его основной язык в 1:N — Python. Все внешние ключи соответствуют существующим записям.

Перенос запросов на предметную область

A1 выводит пары библиотек и основных языков с сортировкой по названию языка. При совпадении языка используется название библиотеки. A2 суммирует число модулей по основному языку и сортирует суммы по убыванию; при равенстве используется название языка. Пустые группы исключены, как в примере преподавателя.

A3 выбирает языки, содержащие отдельное слово «язык», и выводит списки связанных библиотек через таблицу M:N. В шаблоне условия «сотрудник», «отдел», «зарплата» и слово «отдел» заменены соответственно библиотекой, языком, числом модулей и словом «язык».

Реализация и запуск

Для соединений применены list comprehensions, для A3 — dict comprehension. Используются sorted, sum, any и itemgetter. Данные заданы в памяти, внешние библиотеки и подключение к сети не требуются. Python 3.10 или новее.

```
python3 rk_01/main.py
```

Команда выполняется из корня pcpl-labs.

Результаты выполнения

```
$ python3 rk_01/main.py
РК-1. Вариант 22. Запросы А
А1. Библиотеки и языки, сортировка по языкам
('Boost', 6, 'язык C++')
('NumPy', 8, 'язык Python')
('Polars', 12, 'язык Python')
('serde', 4, 'язык Rust')

А2. Суммарное число модулей по языкам, по убыванию
('язык Python', 20)
('язык C++', 6)
('язык Rust', 4)

А3. Слово 'язык' в названии, связь M:N
язык Python: Polars, NumPy
язык Rust: Polars, serde
язык C++: Boost
```

Рисунок 1. Экранная форма фактического вывода программы

Изображение построено по сохранённому stdout и stderr. Полные журналы приведены в папке results.

Проверка и вывод

Суммарное число модулей: Python — 20, C++ — 6, Rust — 4. В А3 для Rust выводятся Polars и serde, что подтверждает использование связи M:N. Все три результата сопоставлены с функциями РК-2 скриптом tools/verify_extra.py; расхождений нет.

Источник задания

Условия РК-1, таблица вариантов, строка 22 и запросы А. Проверено 19 октября 2026 года.

https://github.com/ugapanyuk/courses_current/blob/main/python/rk/pcpl/pcpl_rk_1.pdf

Текст программы

```
# Файл main.py

"""РК-1. Вариант 22, запросы А. Запуск: python3 rk_01/main.py."""

from operator import itemgetter


class Library:
    """Библиотека: ключ, название, число модулей, основной язык."""

    def __init__(self, id, name, module_count, language_id):
        self.id = id
        self.name = name
        self.module_count = module_count
        self.language_id = language_id


class Language:
    """Язык программирования на стороне «один»."""

    def __init__(self, id, name):
        self.id = id
        self.name = name


class LibraryLanguage:
    """Связь многие-ко-многим по идентификаторам."""

    def __init__(self, library_id, language_id):
        self.library_id = library_id
        self.language_id = language_id


def main():
    # Учебные данные: число модулей задано условно, не взято из релизов.
    # Основной язык задаёт учебную связь 1:N; Polars доступен Python и Rust.
    languages = [
        Language(1, "язык Python"),
        Language(2, "язык Rust"),
        Language(3, "язык C++"),
    ]
    libraries = [
        Library(1, "Polars", 12, 1),
        Library(2, "NumPy", 8, 1),
        Library(3, "Boost", 6, 3),
        Library(4, "serde", 4, 2),
    ]
    links = [
        LibraryLanguage(1, 1),
        LibraryLanguage(1, 2),
        LibraryLanguage(2, 1),
        LibraryLanguage(3, 3),
        LibraryLanguage(4, 2),
    ]

    one_to_many = [
```

Текст программы. Продолжение

```
        (lib.name, lib.module_count, lang.name)
        for lang in languages
        for lib in libraries
        if lib.language_id == lang.id
    ]
    many_to_many = [
        (lib.name, lib.module_count, lang.name)
        for link in links
        for lib in libraries
        for lang in languages
        if lib.id == link.library_id and lang.id == link.language_id
    ]

    print("РК-1. Вариант 22. Запросы А")
    print("А1. Библиотеки и языки, сортировка по языкам")
    for row in sorted(one_to_many, key=itemgetter(2, 0)):
        print(row)

    print("\nА2. Суммарное число модулей по языкам, по убыванию")
    totals = [
        (lang.name, sum(row[1] for row in one_to_many
                        if row[2] == lang.name))
        for lang in languages
        if any(row[2] == lang.name for row in one_to_many)
    ]
    for row in sorted(totals, key=lambda row: (-row[1], row[0])):
        print(row)

    print("\nА3. Слово 'язык' в названии, связь M:N")
    result = {
        lang.name: [name for name, _, language in many_to_many
                    if language == lang.name]
        for lang in languages
        if "язык" in lang.name.casefold().split()
    }
    for language, names in result.items():
        print(f"{language}: {', '.join(names)}")


if __name__ == "__main__":
    main()
```